VOD2Video: AI-Powered Highlight Detection and VOD Processing Pipeline

Course: Deep Learning / Machine Learning Final Project
Team Members: Kurt Jensen, Ian Wood, Drew Peterson
Date: April 2026

Github: https://github.com/KurtJJensen01/VOD2Video

Artifacts folder: VOD2Video_Final_Delivery.zip

1. Project Overview

VOD2Video is an AI-powered video processing project designed to turn long livestream recordings into shorter YouTube-style highlight videos. The system takes a raw VOD, splits it into fixed-length clips, scores each clip with a trained model, ranks likely highlight moments, selects the best clips, and assembles them into a final condensed highlight video.

The problem this project focuses on is the amount of time it takes to manually review long livestream recordings. A stream can be an hour or several hours long, and finding the best moments manually is slow and repetitive. VOD2Video automates the first major part of that editing process by identifying clips that are likely to be interesting, exciting, funny, or worth reviewing.

The final project pipeline is:

```
raw VOD -> 5-second clips -> feature extraction / frame and audio loading ->
CNN+LSTM+audio model scoring -> ranked clips -> Phase 7 clip selection ->
Phase 8 final MP4 assembly with teaser
```

For the machine learning part of the project, the task is binary highlight detection. Each labeled 5-second clip is classified as either:

```
1 = highlight
0 = non-highlight
```

By the end of the project, the full demo pipeline was complete through Phase 8. The final output was a 10-minute highlight video generated from a longer VOD using the trained model, clip selection, and video assembly pipeline.

2. Final Project Status

The project is complete through Phase 9.

Phase	Status	Description

Phase	Status	Description
Phase 5A	Complete	Final model lock and hyperparameter search
Phase 6A	Complete	New VOD inference pipeline
Phase 7	Complete	Highlight clip selection package
Phase 8	Complete	Final video assembly with optional teaser intro
Phase 9	Complete	Final report and delivery

The final workflow starts with a raw VOD and ends with a generated MP4 highlight video. Phase 6 splits the VOD into clips and scores them. Phase 7 selects the clips that should go into the final recap. Phase 8 assembles those selected clips into one final video and optionally adds a teaser intro.

The final demo artifact is:

`artifacts/final_video/phase_8_final_demo_vod_10min_v2/final_highlight_video.mp4`

The final video duration is about **608 seconds**, or about **10 minutes and 8 seconds**. It contains a teaser intro made from the loudest moments of selected clips, followed by a black separator, then the main highlight video.

3. Dataset and Preprocessing

The dataset is made up of labeled 5-second clips from livestream VODs. The project uses four labeled CSV sources listed in `datasets.json`.

Source CSV	Total Clips	Highlights	Non-Highlights
`labeling_test/1_Jynxzi_Labels.csv`	500	76	424
`labeling_test/2_Burnt_Peanut_Labels.csv`	500	81	419
`labeling_test/3_Jynxzi_Labels.csv`	50	50	0
`labeling_test/4_Burnt_Peanut_Labels.csv`	131	131	0
Total	**1,181**	**338**	**843**

The final split counts were:

Split	Total Clips	Highlights	Non-Highlights
Train	848	259	589
Validation	166	39	127
Test	167	40	127

The project keeps clip length fixed at 5 seconds. The split process uses a time-aware strategy where nearby clips are grouped together to reduce leakage between training, validation, and test sets. This matters because clips next to each other in a VOD are often visually and contextually similar.

Important columns used by the dataset and pipeline include:

```
vod_id
segment_id
clip_name
clip_path
start_time_seconds
end_time_seconds
start_time_hhmmss
end_time_hhmmss
label
resolved_clip_path
unique_id
split
```

The final feature manifest for the completed Phase 5A run is:

`artifacts/features/phase_5a_final_848_166_167/clip_features.csv`

Earlier draft notes mentioned an older 1,000-row feature manifest. That is no longer the final source of truth for the completed report.

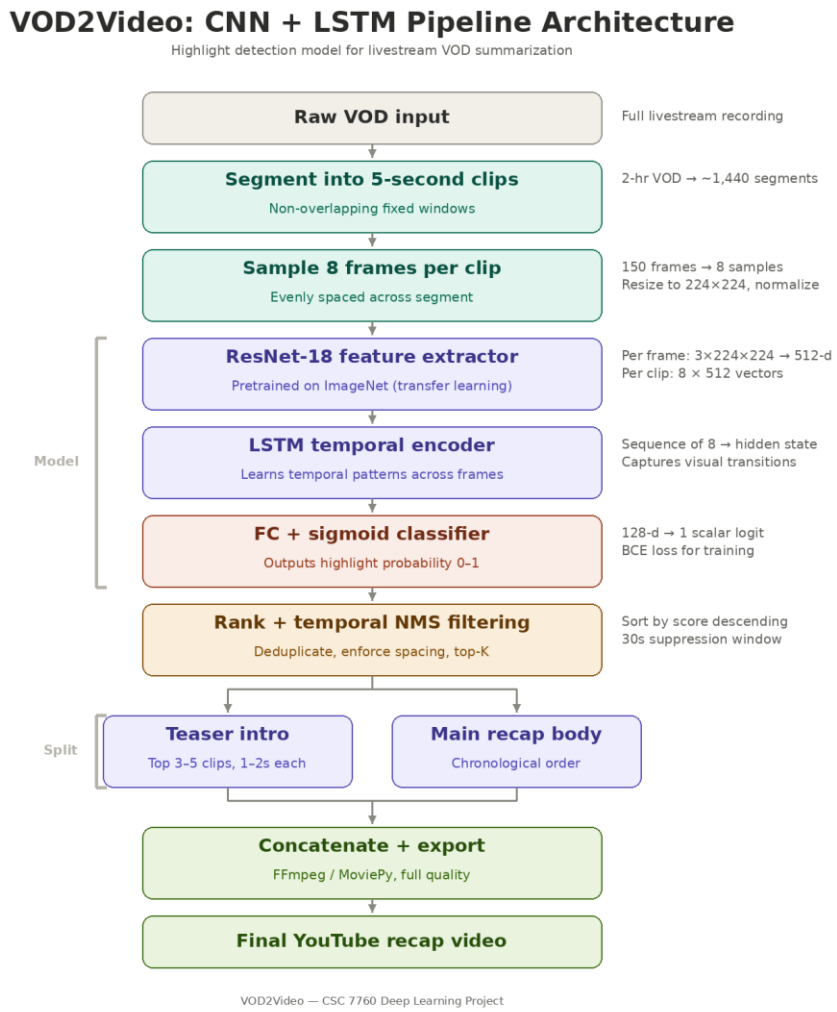
4. Model Architecture

The final model is the **CNN+LSTM+audio/MLP highlight classifier**. In the code, this model is named:

`cnn_lstm_audio`

The model combines visual frame information with audio summary features. The architecture is:

1. A frozen pretrained ResNet18 frame encoder
2. A 1-layer LSTM temporal encoder
3. Seven audio summary features
4. An MLP classification head
5. A binary output for highlight vs. non-highlight



4.1 CNN Frame Encoder

The model uses a pretrained ResNet18 as the frame encoder. The final ResNet classification layer is removed, so the network acts as a visual feature extractor. Each sampled frame becomes a 512-dimensional embedding.

The ResNet18 backbone is frozen by default. This keeps training more manageable and lets the model use pretrained visual features without needing a much larger dataset.

4.2 Frame Sampling

Each 5-second clip is represented by 10 sampled frames. Each frame is resized to 224 by 224 before passing through ResNet18.

```
frame_count = 10
frame_size = 224
```

This gives the LSTM a sequence of frame embeddings rather than one static image.

4.3 LSTM Temporal Encoder

The LSTM models how the sampled frames change over time inside the clip. This matters because a highlight is often based on motion, reaction, or a short sequence of events, not just one frame.

Final LSTM settings:

```
LSTM hidden size = 256
LSTM layers = 1
```

The final LSTM output becomes the video-level embedding for the clip.

4.4 Audio Features

The model also uses seven audio summary features. Audio is important because highlights often include loud reactions, sudden noises, laughing, yelling, or other energy changes.

The final feature rows had audio available for all 1,181 clips. If audio decoding fails in the pipeline, the code can fall back to zero audio features, but the final run used audio features.

4.5 MLP Classification Head

The LSTM video embedding is concatenated with the seven audio features and passed into an MLP classifier.

The MLP head is structured as:

```
Linear(263 -> 128)
ReLU
Dropout(0.5)
Linear(128 -> 64)
ReLU
Dropout(0.5)
Linear(64 -> 1)
```

The model outputs one logit. A sigmoid converts that logit into a probability. The final decision threshold is applied to classify the clip as highlight or non-highlight.

4.6 Loss Function and Class Imbalance

The model uses `BCEWithLogitsLoss`. Positive class weighting is used to help with class imbalance. The final positive class weight was:

```
pos_weight = 2.2741
```

This helps the model pay more attention to highlight clips because non-highlights are more common in the dataset.

5. Training and Hyperparameter Search

The main hyperparameter search script was:

`tools/run_hyperparameter_search.py`

The final search evaluated 36 training combinations.

The search grid included:

Hyperparameter	Values
Epochs	50, 75, 100
Learning rate	0.001, 0.0003, 0.0001, 0.00005
Batch size	4, 8, 16
Weight decay	0.0001
Patience	15
Dropout	0.5
LSTM hidden dimension	256
Audio feature dimension	7
Backbone	Frozen ResNet18

The final selected run was:

`e100_lr0.0003_bs8`

This run was selected because it had the best validation F1 score from the hyperparameter search.

Final selected settings:

Setting	Value
Epoch budget	100
Best epoch	39
Learning rate	0.0003
Batch size	8
Weight decay	0.0001
Patience	15
Threshold	0.60
Device	CUDA

Final checkpoint:

`artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/best_model.pt`

Core Phase 5A result files:

```
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/result.json
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/metrics.json
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/training_history.json
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/val_predictions.csv
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/test_predictions.csv
```

6. Final Model Results

6.1 Validation Results

Metric	Value
Accuracy	0.8012
Precision	0.5714
Recall	0.6154
F1	0.5926
True Positives	24
True Negatives	109
False Positives	18
False Negatives	15

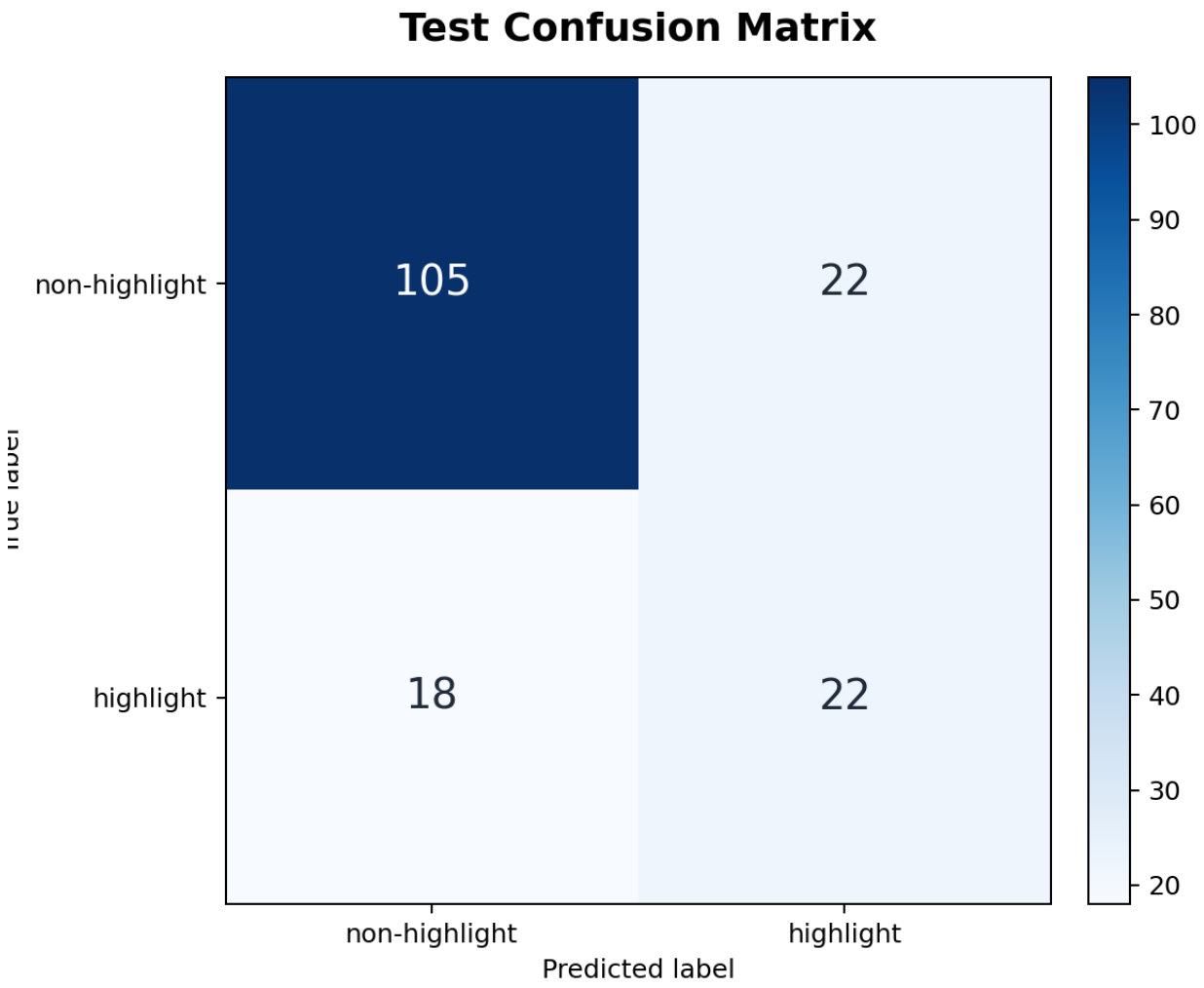
6.2 Test Results

Metric	Value
Accuracy	0.7605
Precision	0.5000
Recall	0.5500
F1	0.5238
True Positives	22
True Negatives	105
False Positives	22
False Negatives	18

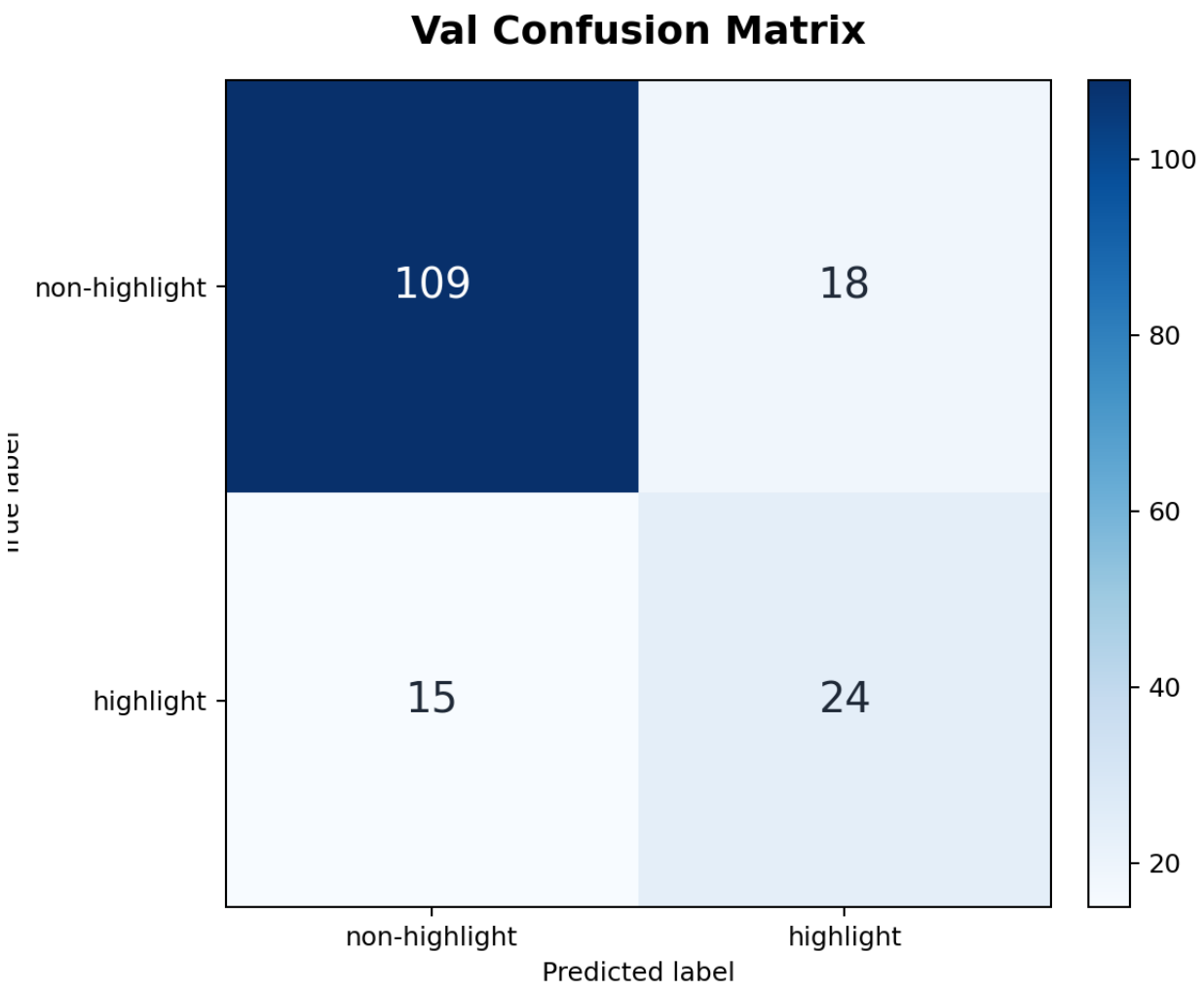
The test F1 score shows that the model learned useful signal, but it is not perfect. This makes sense for the problem. Highlight detection is subjective, the dataset is still small for deep video learning, and a 5-second clip may not always provide enough context to know whether something is truly a highlight.

The final report-ready visual artifacts are:

```
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/test_confusion_matrix.png
```



artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/val_confusion_matrix.png

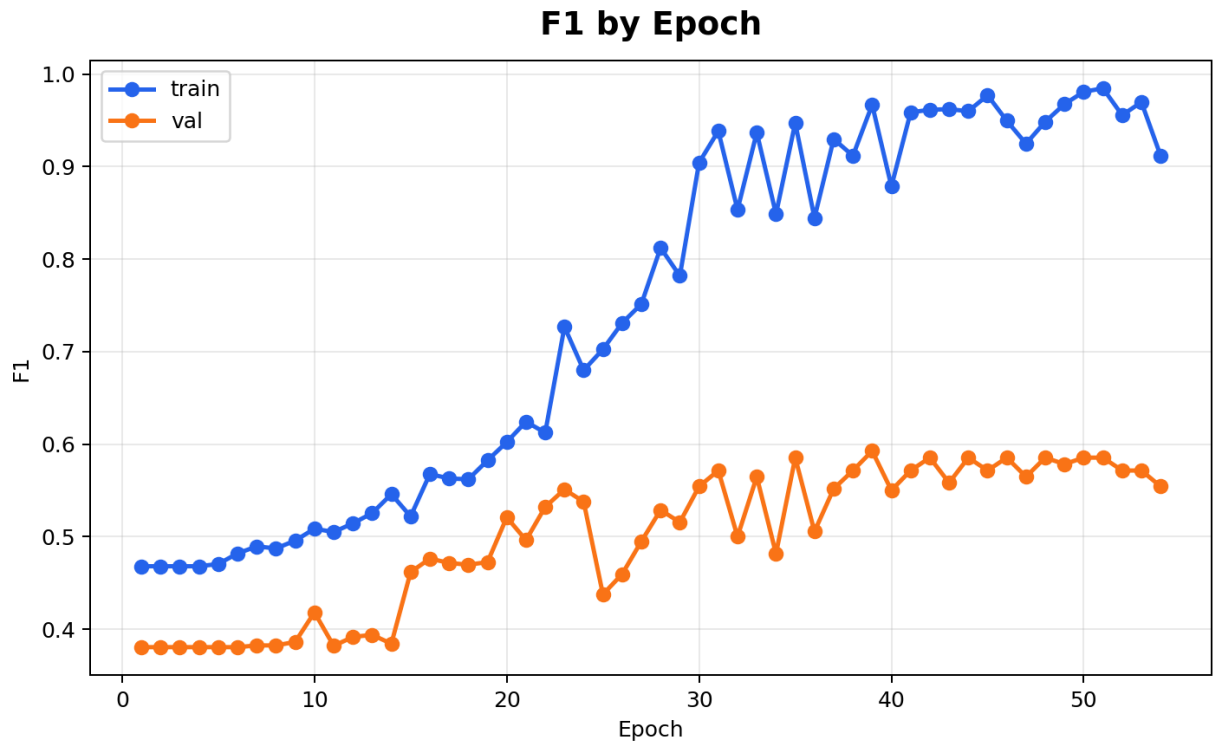


artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/best_epoch_metrics_table.png

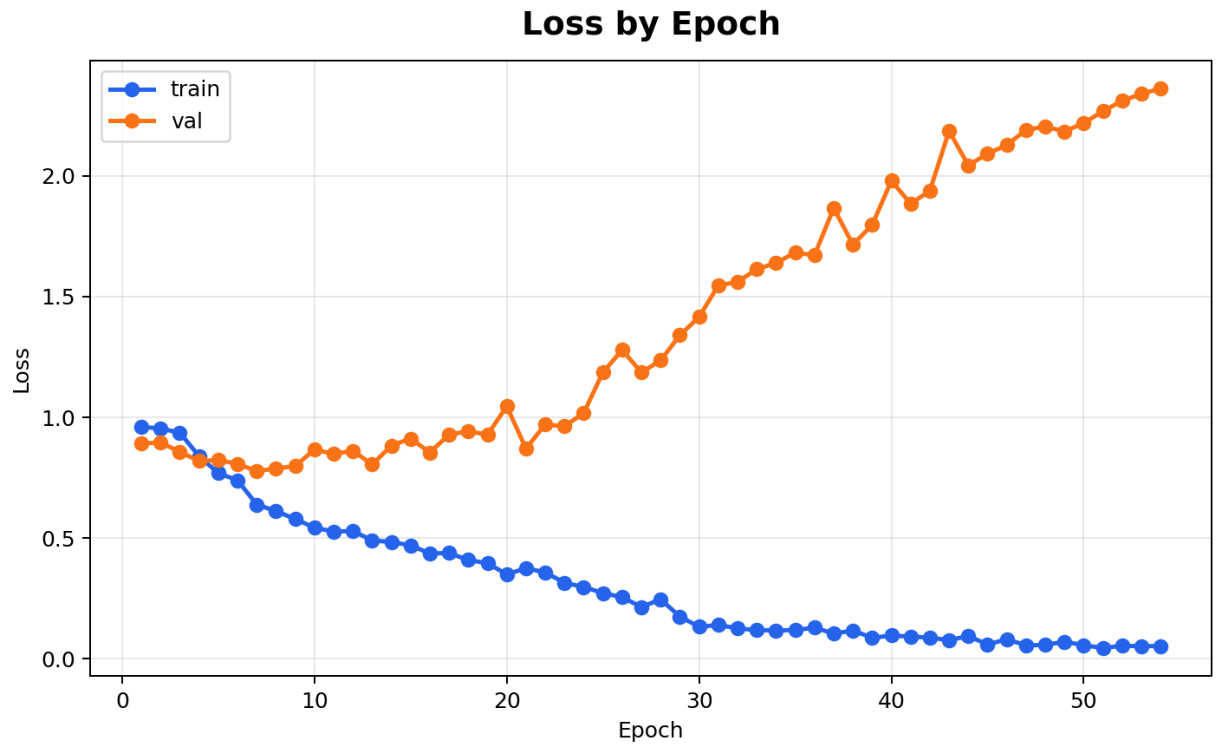
Best Epoch Metrics (epoch 39)

split	accuracy	precision	recall	f1
train	0.980	0.976	0.958	0.967
val	0.801	0.571	0.615	0.593
test	0.760	0.500	0.550	0.524

artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/f1_by_epoch.png



artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/loss_by_epoch.png

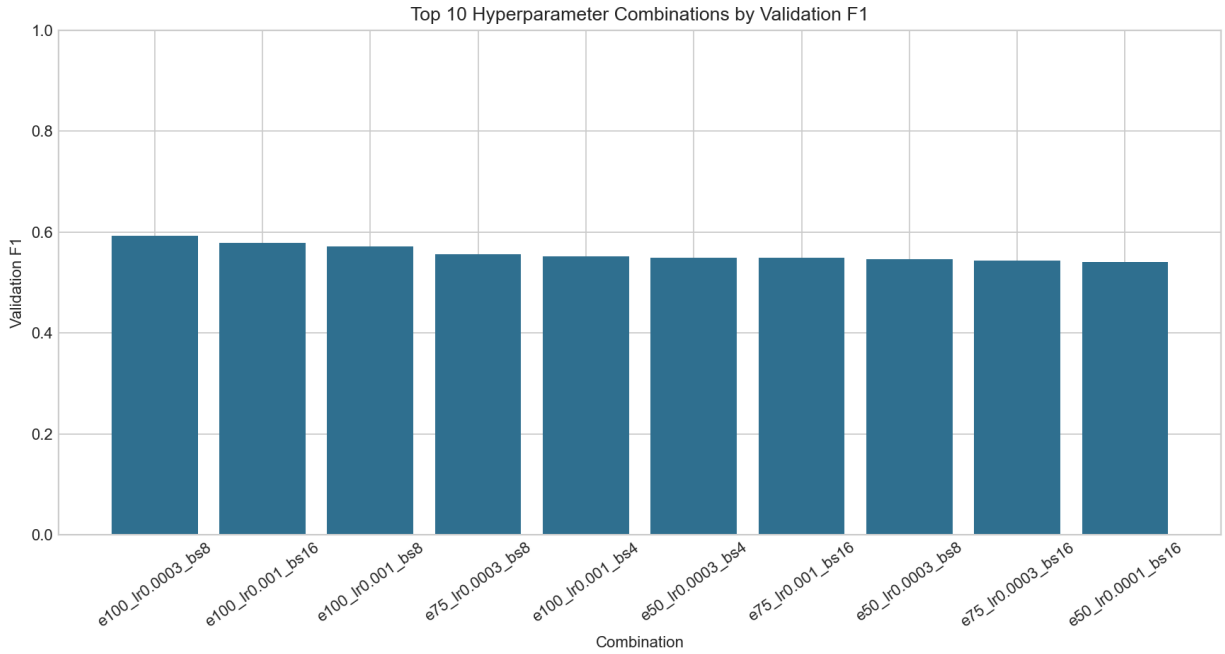


artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/threshold_sweep_table.png

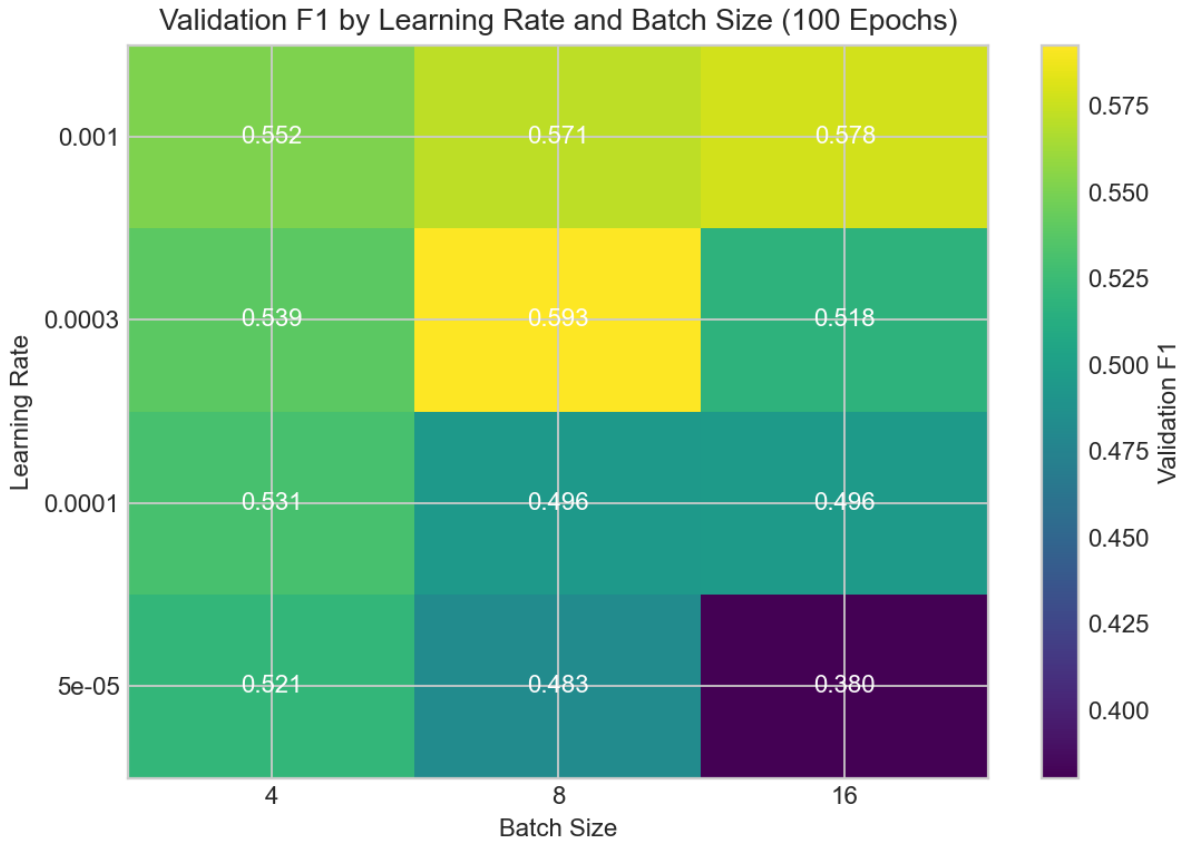
Validation Threshold Sweep

threshold	accuracy	precision	recall	f1	tp	tn	fp	fn	sample_count
0.05	0.717	0.429	0.615	0.505	24	95	32	15	166
0.10	0.729	0.444	0.615	0.516	24	97	30	15	166
0.15	0.759	0.490	0.615	0.545	24	102	25	15	166
0.20	0.771	0.511	0.615	0.558	24	104	23	15	166
0.25	0.777	0.522	0.615	0.565	24	105	22	15	166
0.30	0.777	0.522	0.615	0.565	24	105	22	15	166
0.35	0.783	0.533	0.615	0.571	24	106	21	15	166
0.40	0.789	0.545	0.615	0.578	24	107	20	15	166
0.45	0.795	0.558	0.615	0.585	24	108	19	15	166
0.50	0.795	0.558	0.615	0.585	24	108	19	15	166
0.55	0.795	0.558	0.615	0.585	24	108	19	15	166
0.60	0.801	0.571	0.615	0.593	24	109	18	15	166
0.65	0.789	0.550	0.564	0.557	22	109	18	17	166
0.70	0.789	0.550	0.564	0.557	22	109	18	17	166
0.75	0.795	0.564	0.564	0.564	22	110	17	17	166
0.80	0.795	0.568	0.538	0.553	21	111	16	18	166
0.85	0.789	0.559	0.487	0.521	19	112	15	20	166
0.90	0.783	0.548	0.436	0.486	17	113	14	22	166
0.95	0.807	0.630	0.436	0.515	17	117	10	22	166

`artifacts/hyperparameter_search_broad/plots/top_10_val_f1.png`



artifacts/hyperparameter_search_broad/plots/val_f1_heatmap_e100.png



Example Clips for Each Outcome

Outcome	Clip ID	Predicted probability	Note
True Positive	3_seg_00494	0.998	Confident correct highlight
True Negative	1_seg_03831	0.006	Confident correct non-highlight

Outcome	Clip ID	Predicted probability	Note
False Positive	2_seg_03442	0.989	Confident mistake
False Negative	2_seg_01983	0.009	Confident miss

7. Evaluation Method

The project evaluates binary classification performance using:

```
accuracy
precision
recall
F1 score
true positives
true negatives
false positives
false negatives
confusion matrix
```

Accuracy is useful, but it is not enough by itself because the dataset is imbalanced. A model could look accurate by mostly predicting non-highlight. For this project, F1, precision, and recall are more important because the goal is to find useful highlights.

The validation threshold was selected through a threshold sweep. The final model used a threshold of 0.60 for classification evaluation. For the final 10-minute demo video, Phase 7 used a lower threshold of 0.35 so the pipeline could produce a longer highlight video from an unlabeled VOD. That selection threshold was used for the demo length target, not for reporting labeled test-set model performance.

8. Phase 6: New VOD Inference Pipeline

Phase 6 runs the trained model on a new unlabeled VOD. This is where the project moves from labeled evaluation into an actual demo workflow.

Phase 6 does the following:

1. Takes a raw VOD as input
2. Splits the VOD into 5-second clips
3. Creates a clip manifest
4. Extracts or loads clip features
5. Runs the CNN+LSTM+audio model
6. Scores each clip
7. Writes ranked prediction outputs

Final Phase 6 output folder:

```
artifacts/inference/phase_6_final_demo_vod/
```

Phase 6 scored **933 clips** from the final demo VOD. At the 0.60 threshold, **801 clips** were predicted as highlights. This showed that the model was useful for ranking clips, but it also showed that the model may be over-selecting highlights on unseen VODs. That is discussed as a limitation.

Final Phase 6 output files:

```
artifacts/inference/phase_6_final_demo_vod/clip_manifest.csv
artifacts/inference/phase_6_final_demo_vod/features/clip_features.csv
artifacts/inference/phase_6_final_demo_vod/features/feature_summary.json
artifacts/inference/phase_6_final_demo_vod/inference/scored_clips.csv
artifacts/inference/phase_6_final_demo_vod/inference/top_highlights.csv
artifacts/inference/phase_6_final_demo_vod/inference/inference_summary.json
artifacts/inference/phase_6_final_demo_vod/pipeline_summary.json
```

The most important file for the next phase is:

```
artifacts/inference/phase_6_final_demo_vod/inference/scored_clips.csv
```

9. Phase 7: Highlight Clip Selection

Phase 7 takes the scored clips from Phase 6 and creates a final selected clip package for video assembly.

Phase 7 does the following:

1. Reads `scored_clips.csv`
2. Filters clips based on a probability threshold
3. Applies a top-k limit

4. Optionally applies a minimum time-gap filter
5. Copies the selected clips into a clean selected clips folder
6. Writes a selected highlights manifest
7. Writes a selection summary JSON file

For the final 10-minute demo, the selected settings were:

Setting	Value
Threshold	0.35
Top-k	120
Minimum gap	0.0 seconds
Selected clips	120

The threshold was lowered from 0.60 to 0.35 for the demo video because the goal was to create an approximately 10-minute output. Since each clip is 5 seconds, 120 clips gives about 600 seconds of main video content.

Final Phase 7 folder:

```
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/
```

Final Phase 7 outputs:

```
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/selected_highli
ghts_manifest.csv
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/selection_summa
ry.json
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/selected_clips/
```

10. Phase 8: Final Video Assembly

Phase 8 assembles the selected clips from Phase 7 into a final MP4 highlight video.

Phase 8 does the following:

1. Reads the Phase 7 selected highlights manifest
2. Resolves selected clip paths
3. Sorts the clips chronologically
4. Creates an FFmpeg concat list
5. Merges the clips into one MP4
6. Optionally creates a teaser intro
7. Writes the final video and assembly artifacts

The final video is:

```
artifacts/final_video/phase_8_final_demo_vod_10min_v2/final_highlight_video.m
p4
```

Final video details:

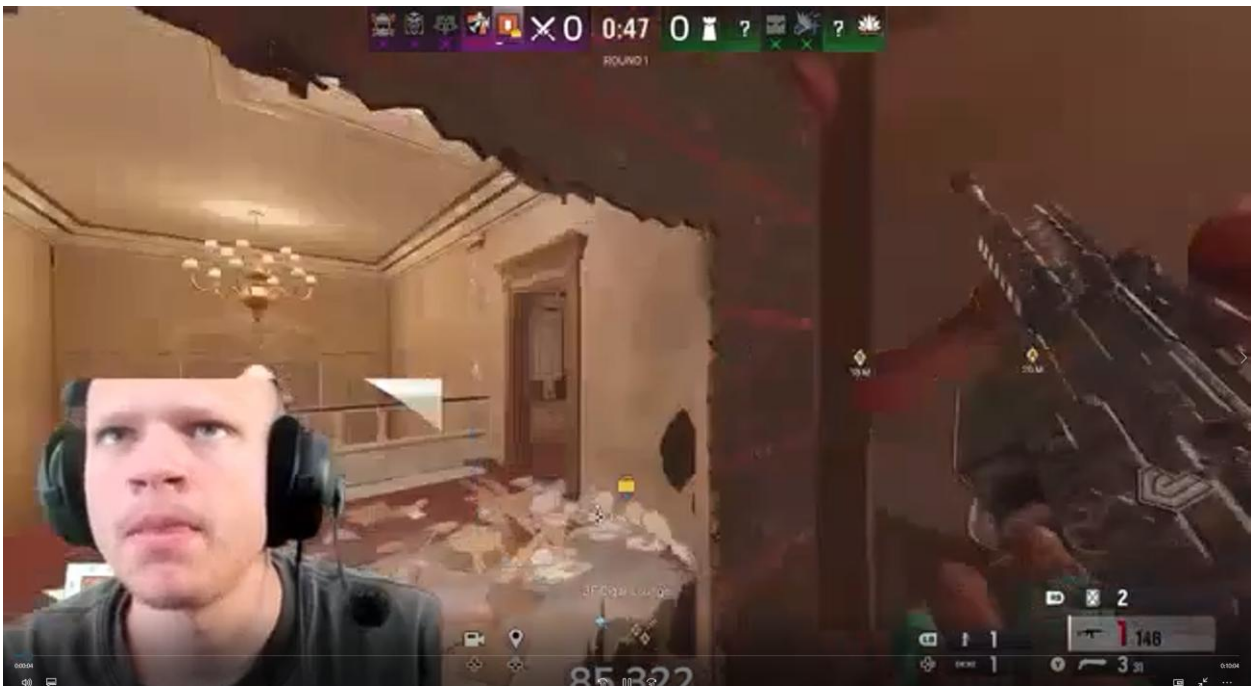
Item	Value
Final duration	608.066 seconds, about 10:08
Selected clips	120
Main clip duration	600 seconds
Assembly order	Chronological
Teaser enabled	Yes
Teaser snippets	5
Teaser snippet length	1.5 seconds
Teaser order	Score
Teaser snippet mode	Loudest
Separator	0.5 seconds black screen

The teaser starts the video with short loudest-moment snippets from selected clips. After the teaser, the video uses a short black separator to clearly transition into the main highlight video.

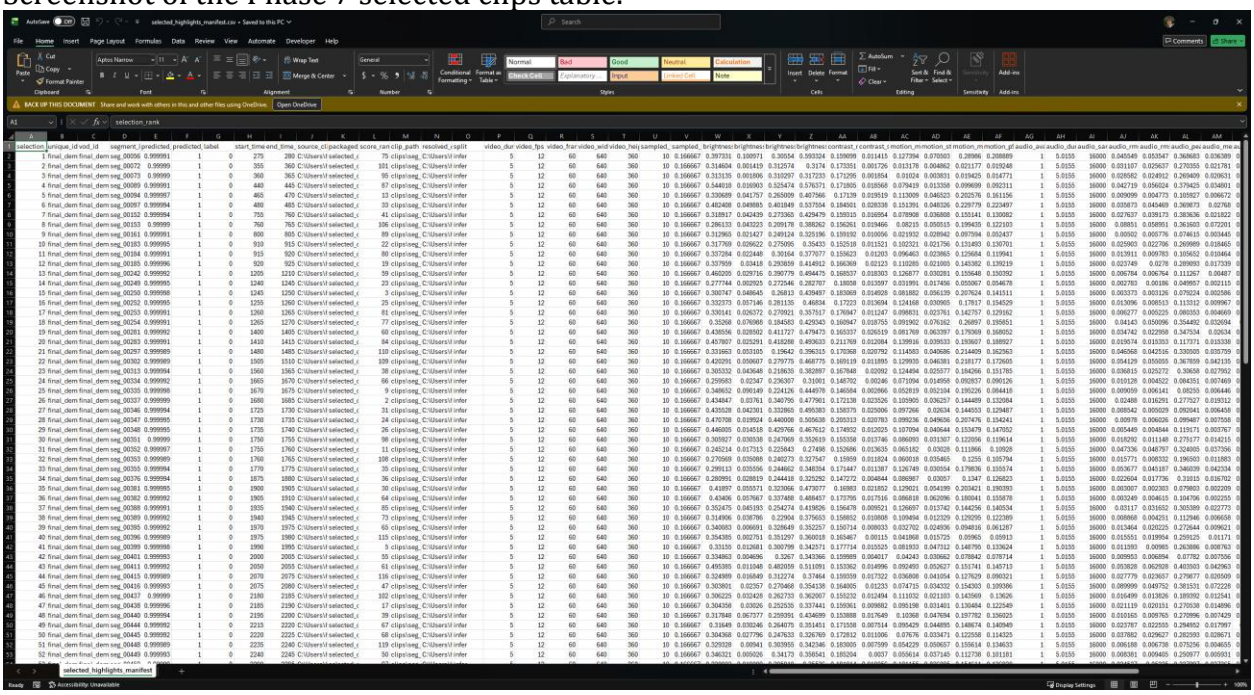
Final Phase 8 outputs:

```
artifacts/final_video/phase_8_final_demo_vod_10min_v2/final_highlight_video.m
p4
artifacts/final_video/phase_8_final_demo_vod_10min_v2/assembly_manifest.csv
artifacts/final_video/phase_8_final_demo_vod_10min_v2/assembly_summary.json
artifacts/final_video/phase_8_final_demo_vod_10min_v2/concat_list.txt
artifacts/final_video/phase_8_final_demo_vod_10min_v2/teaser_summary.json
artifacts/final_video/phase_8_final_demo_vod_10min_v2/teaser_concat_list.txt
artifacts/final_video/phase_8_final_demo_vod_10min_v2/teaser_clips/
artifacts/final_video/phase_8_final_demo_vod_10min_v2/teaser_separator.mp4
artifacts/final_video/phase_8_final_demo_vod_10min_v2/main_highlight_video.mp
4
artifacts/final_video/phase_8_final_demo_vod_10min_v2/main_highlight_video_st
andardized.mp4
```

Screenshot from the final video.



Screenshot of the Phase 7 selected clips table.



11. Commands Used

11.1 Phase 5A Visuals

```
python tools/visualize_cnn_run.py --run-dir
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8 --output-dir
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8
```

11.2 Phase 6 Final Inference

```
python tools/run_vod_inference_pipeline.py --input
"C:\Users\hocke\Downloads\YTDown_YouTube_Bernard-Picks-My-Operator-in-
Rainbow-Six_Media_m8iJCE-o29A_001_1080p.mp4" --vod-id final_demo_vod --
output-dir artifacts/inference/phase_6_final_demo_vod --checkpoint
artifacts/hyperparameter_search_broad/e100_lr0.0003_bs8/best_model.pt --
threshold 0.60 --segment-length 5 --top-k 150 --min-time-distance-seconds 20
--ffmpeg-path C:\ffmpeg\bin\ffmpeg.exe --ffprobe-path
C:\ffmpeg\bin\ffprobe.exe
```

11.3 Phase 7 10-Minute Selection

```
python tools/select_highlight_clips.py --input
artifacts/inference/phase_6_final_demo_vod/inference/scored_clips.csv --
output-dir artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2 --
threshold 0.35 --top-k 120 --min-gap-seconds 0
```

11.4 Phase 8 Final Assembly With Teaser

```
python tools/assemble_final_video.py --selection-manifest
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/selected_highli
ghts_manifest.csv --selection-dir
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2 --output-dir
artifacts/final_video/phase_8_final_demo_vod_10min_v2 --output-name
final_highlight_video.mp4 --order chronological --ffmpeg-path
C:\ffmpeg\bin\ffmpeg.exe --include-teaser --teaser-clip-count 5 --teaser-
snippet-seconds 1.5 --teaser-order score --teaser-snippet-mode loudest --
teaser-transition-seconds 0.5
```

12. Limitations

There are several limitations in the current version of the project.

First, the dataset is still small for a deep video learning problem. The final labeled dataset has 1,181 clips, which is useful for a course project but still limited for training a robust video model.

Second, the labels are subjective. A highlight can depend on context, humor, player reaction, or what the viewer personally finds interesting. This makes the task harder than a more objective classification problem.

Third, the model only looks at 5-second clips. Some moments need longer context to understand why they matter.

Fourth, the model may be over-selecting highlights on new VODs. On the final unlabeled VOD, it predicted 801 out of 933 clips as highlights at the 0.60 threshold. This does not affect the labeled test metrics, but it shows the probabilities may need better calibration before the system is used as a fully automatic product.

Fifth, the final VOD does not have human labels. This means the final video quality is evaluated qualitatively by watching the output, not with accuracy or F1.

Sixth, the final 10-minute video required lowering the Phase 7 selection threshold to 0.35. That is reasonable for creating a longer demo, but it means the demo selection settings are different from the model evaluation threshold.

Seventh, the final video assembly is simple. It merges clips and adds a teaser, but it does not yet include advanced editing, captions, sound leveling, transitions between every clip, or manual review controls.

13. Future Work

Future work should focus on improving both the model and the product pipeline.

The biggest improvement would be adding more labeled data from more VODs, streamers, and games. More data would help the model generalize better.

The model could also benefit from better probability calibration. Since the model predicted many positives on the final VOD, future work should improve thresholding and confidence calibration.

The clip selection logic could be improved with better redundancy controls. For example, the system could avoid selecting too many clips from the same small section of the VOD while still allowing enough clips to reach a target video length.

A manual review interface would also be useful. Instead of automatically accepting all selected clips, the user could approve, remove, or reorder clips before final assembly.

Future video assembly improvements could include smoother transitions, captions, audio leveling, intro title cards, outro cards, and more polished teaser editing.

A desktop app would make the system easier to use. The user could select a VOD, choose an output length, run the model, review selected clips, and export the final video without using terminal commands.

14. Conclusion

VOD2Video demonstrates an end-to-end machine learning pipeline for automatically finding and assembling highlights from livestream VODs. The project includes dataset loading, time-aware splitting, CNN+LSTM+audio modeling, hyperparameter search, evaluation, new-VOD inference, clip selection, and final video assembly.

The final selected model was the `e100_lr0.0003_bs8` CNN+LSTM+audio run. It achieved a validation F1 score of 0.5926 and a test F1 score of 0.5238. These results are not perfect, but they show the model learned useful highlight signals from both video frames and audio features.

The full demo pipeline was completed successfully. A raw VOD was processed into 933 scored clips, Phase 7 selected 120 clips for a 10-minute output, and Phase 8 assembled those clips into a final highlight video with a teaser intro. The final output video was about 10 minutes and 8 seconds long.

Overall, the project shows a realistic first version of an automated VOD-to-highlight workflow. It is not yet a fully polished product, but it successfully proves the main idea: a long livestream recording can be processed by an AI model and turned into a shorter highlight video with minimal manual editing.

Appendix A: Final Report-Ready Artifact Paths

```
Final confusion matrix:
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/test_confusion_matrix.png

Final metrics table:
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/best_epoch_metrics_table.png

Training graphs:
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/f1_by_epoch.png
artifacts/visualization/phase_5a_final_e100_lr0.0003_bs8/loss_by_epoch.png

Hyperparameter comparison:
artifacts/hyperparameter_search_broad/plots/top_10_val_f1.png
artifacts/hyperparameter_search_broad/results.csv

Phase 6 final inference:
artifacts/inference/phase_6_final_demo_vod/inference/scored_clips.csv
artifacts/inference/phase_6_final_demo_vod/inference/top_highlights.csv

Phase 7 final selected clips:
artifacts/highlight_selection/phase_7_final_demo_vod_10min_v2/selected_highlights_manifest.csv

Phase 8 final video:
artifacts/final_video/phase_8_final_demo_vod_10min_v2/final_highlight_video.mp4
```

Appendix B: Artifacts to Avoid Using as Final Evidence

The following folders are stale, smoke tests, or older outputs and should not be referenced as final report evidence:

```
artifacts/inference/phase_6_test_3min/
artifacts/highlight_selection/phase_7_test_3min/
```

```
artifacts/final_video/phase_8_test_3min/
artifacts/final_video/phase_8_test_no_teaser/
artifacts/final_video/phase_8_test_with_teaser/
artifacts/highlight_selection/phase_7_final_demo_vod/
artifacts/highlight_selection/phase_7_final_demo_vod_10min/
artifacts/final_video/phase_8_final_demo_vod/
artifacts/Archive/
outputs/phase_5a_final_metrics_package.md
outputs/phase_5b_final_demo_examples.md
```

Generated videos, checkpoints, raw VODs, generated clips, and artifact folders should not be committed to GitHub unless the submission specifically asks for them.